

FabMind Agent

Final 10/10 GitHub + Antigravity + Codex Execution Playbook

반도체 Load Port / FOUP Clamp / EtherCAT I/O 전용 근거 기반 Agentic AI 트러블슈팅 플랫폼

항목	최종 기준
프로젝트 성격	졸업작품 + 취업 포트폴리오 + 현업형 제품 설계
핵심 범위	Load Port / FOUP Clamp / EtherCAT I/O
AI 원칙	LLM 의존이 아니라 deterministic rule + evidence 기반
보안 원칙	내부망, 읽기 전용, 데이터 무반출, 사람 승인
GitHub 목표	README, CI, PR 기록, screenshots 로 회사가 평가 가능

1. 냉정한 최종 피드백

방향은 맞다. 다만 “세상에 없는 완벽한 시스템”이라는 표현은 발표에서 그대로 쓰면 위험하다. 검증 가능한 표현은 “반도체 현장 제약을 반영한 독자적 조합의 근거 기반 트러블슈팅 시스템”이다. 완전한 세계 최소를 증명할 필요는 없다. 평가자는 독창성보다 문제 정의, 제약 인식, 구현 완성도, 시연 안정성, 코드 품질을 본다.

10 점 프로젝트가 되려면 기능을 무작정 늘리지 말고, 하나의 장비군에 대해 끝까지 닫힌 흐름을 만들어야 한다. 따라서 최종 범위는 Load Port / FOUP Clamp / EtherCAT I/O 로 고정한다. 이 범위 안에서 진단 입력, 근거 검색, 원인 후보, 점검 절차, 보고서, 승인, 감사 로그, GitHub CI 까지 완성하면 졸업작품과 포트폴리오 모두에서 강하다.

2. 10 점짜리 독자성 정의

FabMind Agent 의 독자성은 AI 가 장비를 직접 제어하는 것이 아니다. 오히려 실제 반도체 현장에서는 그런 접근이 보안과 안전 때문에 막힌다. 독자성은 다음 5 가지를 하나의 제품 흐름으로 묶는 데 있다.

- 1) Situation Snapshot Contract: 알람, EtherCAT 상태, DI/DO, 최근 작업, 로그를 표준 입력으로 정규화한다.
- 2) Safety-Gated Deterministic Triage: 위험 조치를 차단하고 규칙 기반으로 원인 후보를 만든다.
- 3) Evidence Graph: 모든 가설과 점검 단계에 근거를 연결한다.
- 4) Agent Timeline UI: AI 가 어떤 단계로 판단했는지 화면에서 보여준다.
- 5) Human Approval + Audit Trail: 보고서 승인과 감사로그로 책임 추적이 가능하다.

3. 왜 이 선택이 논리적인가

반도체 장비는 회사마다 구조, 알람, I/O 맵, 매뉴얼, 정비 이력이 다르다. 따라서 모든 장비를 자동 분석하는 범용 AI 는 졸업작품으로도, 실제 현업 도입으로도 너무 넓다. 반대로 특정 장비군 하나를 깊게 완성하면 현실성과 완성도가 동시에 생긴다.

또한 실제 장비 데이터를 외부 AI 서버로 보내는 구조는 보안상 어렵다. 그래서 기본 설계는 외부 API 가 아니라 내부망, 읽기 전용, deterministic mode 다. AI 는 최종 판단자나 장비 제어자가 아니라 엔지니어 의사결정 보조자다.

4. 최종 요구정의 핵심

사용자는 장비, 알람 코드, 증상, EtherCAT 상태, DI/DO 상태, 최근 작업, 로그를 입력한다. 시스템은 입력을 Situation Snapshot 으로 정규화하고, 알람 코드표, I/O point, EtherCAT 상태, 매뉴얼 chunk, 정비 이력 근거를 검색한다. 이후 deterministic rule engine 이 원인 후보를 만들고, 위험 조치를 safety guardrail 로 차단한다. 결과는 Agent Analysis 화면에 원인 후보 TOP 3, confidence band, evidence drawer, inspection plan 으로 표시된다. 사용자는 체크리스트를 수행하고 보고서 초안을 생성해 senior approval 을 요청한다. 모든 과정은 audit log 로 남는다.

5. 개발 실행 원칙

Antigravity 와 Codex 를 동시에 사용할 때 가장 위험한 것은 “전체 다 만들어줘”라고 시키는 것이다. 이렇게 하면 폴더가 중복되고, API 계약이 틀어지고, UI 만 예쁜 미완성물이 된다. 반드시 PR 단위로 나눈다.

Antigravity 는 UI, 브라우저 검증, 화면 polish, screenshot artifact 에 집중한다. Codex 는 backend, DB, API contract, deterministic agent, pytest, code review 에 집중한다. GitHub 는 최종 결과물의 신뢰성을 보여주는 포트폴리오 장소로 사용한다.

6. Golden Path

최종 시연은 로그인 → 장비 선택 → 진단 생성 → Agent Analysis → Evidence Drawer → Checklist → Report → Approval → Audit Log 흐름으로 고정한다. 이 흐름이 끊기지 않고 5 분 안에 성공해야 한다. 추가 기능은 Golden Path 가 안정화된 뒤에만 붙인다.

7. 실패 케이스까지 구현해야 10 점

정상 케이스만 구현하면 데모처럼 보인다. 현업형 프로젝트로 보이려면 실패 케이스가 있어야 한다. 근거 부족, 장비-알람 불일치, 위험 조치 요청, EtherCAT 상태 충돌, 권한 부족, 보고서 승인 전 수정, evidence 없는 가설을 모두 처리해야 한다.

8. GitHub 공개 전략

GitHub 에는 단순 코드 저장소가 아니라 회사가 읽을 수 있는 제품형 저장소로 올린다. README 에 문제, 제약, 해결, 아키텍처, screenshots, 실행법, 테스트, CI, 발표 스크립트를 넣는다. PR 기록은 개발 습관을 보여주므로 PR-00 부터 PR-12 까지 단계적으로 남기는 것이 좋다. 실제 회사 데이터는 절대 넣지 않고 synthetic demo data 라고 명시한다.

9. PR 단위 작업표

PR	Owner	목표	완료 기준
PR-00	Antigravity	Monorepo scaffold	web/api/infra 생성, health 200, README
PR-01	Codex	CI/Quality Gate	GitHub Actions, pytest, typecheck, Playwright 준비
PR-02	Codex	DB/Seed	30 alarms, 60 I/O, 20 scenarios, tenant isolation
PR-03	Codex	Auth/RBAC/Audit	field/senior/admin 권한, audit log
PR-04	Antigravity	Design System	industrial SaaS UI shell, 상태 처리
PR-05	Both	Equipment Pages	장비 상세, 알람, I/O, EtherCAT, 문서
PR-06	Codex	Diagnosis API	session create/detail/aggregate, validation
PR-07	Codex	Agent Engine	Scenario A/B/C deterministic tests
PR-08	Antigravity	Agent UI	timeline, hypothesis, evidence drawer, risk banner
PR-09	Both	Checklist	step 상태, note, audit
PR-10	Both	Report/Approval	draft, request, approve/reject, audit
PR-11	Antigravity	Dashboard/Audit Console	상태 추적, 감사 검색
PR-12	Both	Final E2E/Polish	CI green, screenshots, README final

10. 바로 넣을 첫 프롬프트

Antigravity 첫 프롬프트

You are building FabMind Agent. Read AGENTS.md, README.md, docs/00_project_one_page.md, and docs/04_golden_path_spec.md. Do not implement yet. Report the current structure, missing files, violated constraints, and the exact PR task to do next.

Codex 첫 프롬프트

Read AGENTS.md and inspect the repository. Do not implement yet. Report current structure, missing files, violated constraints, next recommended PR, and exact files to modify.

11. GitHub 사용 순서

- 1) GitHub 에서 fabmind-agent 빈 저장소를 만든다.
- 2) ZIP 패키지 내용을 로컬 폴더에 압축 해제한다.
- 3) git init, git add, git commit, git remote add origin, git push 순서로 main 에 올린다.
- 4) 이후에는 PR-00 부터 PR-12 까지 브랜치를 나눠 개발한다.
- 5) README, screenshots, CI 결과, 발표 스크립트를 최종 정리한다.

```
git init
git add .
git commit -m "PR-00 initialize FabMind Agent repository"
git branch -M main
git remote add origin https://github.com/<YOUR_ID>/fabmind-agent.git
git push -u origin main
```

12. 최종 만점 판정

영역	만점 조건
문제 정의	반도체 현장 문제와 보안/호환/안전 제약을 설명
제품 설계	Load Port / FOUP Clamp 범위 고정, 내부망 읽기 전용
Agentic AI	단계별 workflow, deterministic rule, evidence link
UI	Agent Analysis 중심, 상태 처리, 고급 SaaS 느낌
품질	pytest, Playwright, GitHub Actions, PR 기록
발표	5 분 Golden Path 시연, 근거 기반 방어 가능